

**A SINGULAR VALUE DECOMPOSITION APPROACH TO  
PRINCIPLE COMPONENT ANALYSIS**

SEAN MEEHAN

## INTRODUCTION

Principle Component Analysis(PCA) is a fundamental technique in data science, machine learning, and many other fields. Basically, it is a method to reduce the dimensionality of a data set. It does this by projecting that data set, represented as a mean centered, standardized data matrix, onto a new set of axes called principle components. These principle component axes are along the lines of the eigenvectors of the data matrix. The data matrix has as many eigenvectors as there are variables in the data set, so, in principle, this is not strictly a dimension reduction technique. However, we can take only the eigenvectors of the data matrix that have large enough eigenvalues and use only these eigenvalues as a basis upon which to project our matrix.

This process is usually done using the eigendecomposition of the covariance matrix of the data matrix. However, in this paper we will explore an alternate way to perform PCA using Singular Value Decomposition (SVD).

## THEOREMS, RESULTS, AND ALGORITHMS

In this section we will explore the theorems, results, and algorithms that are relevant to computing the PCA of a data set through SVD. Theorems will be offered without proof as this paper takes an applied approach to the problem at hand rather than a theoretical.

**Theorems.**

*Theorem 1.* Let  $A \in M_{m,n}(\mathbb{F})$  have rank  $r$ . Then there exists matrices  $U \in M_m(\mathbb{F})$  and  $V \in M_n(\mathbb{F})$  with  $U$  and  $V$  unitary, or orthogonal(depending on the field) and unique real numbers  $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r \geq 0$  such that:

$$A = U\Sigma V^*,$$

where  $\Sigma \in M_{m,n}(\mathbb{R})$  has  $(j, j)$  entry  $\sigma_j$  for  $1 \leq j \leq r$  and all other entries 0. The  $\sigma_j$  are the singular values of  $A$ . [1]

*Theorem 2.* Let  $A \in M_n(\mathbb{F})$  and  $V$  be a matrix whose columns are the eigenvectors of  $A$ . If  $D$  is a diagonal matrix where the entries are the correspondent eigenvalues for the eigenvectors in  $V$ , then the eigendecomposition of  $A$  is:

$$A = VDV^{-1}.$$

It is conventional to write  $D$  with the eigenvalues in decreasing order along the diagonal, and to adjust the eigenvectors in  $V$  accordingly.[2]

*Theorem 3.* The covariance matrix,  $S$ , of a given mean centered data matrix,  $X$ , can be written:

$$S = \frac{X^T X}{n-1},$$

where  $n$  is the number of observations in  $X$ .[3]

**Results.** We only need one result for the goal of this paper. Result 1 is essentially a corollary of all three theorems, but, since it requires all three, we will call it a result.

*Result 1.* Let  $X$  be our standardized data matrix. We can take the SVD of  $X$ . This gives  $X = U\Sigma V^*$ , as in Theorem 1. So, we have,

$$\begin{aligned} X &= U\Sigma V^* \\ \implies X^T X &= V\Sigma^* U^* \times U\Sigma V^* \\ &= VDV^*, \text{ since } U \text{ is unitary,} \\ \implies \frac{X^T X}{n-1} &= \frac{VDV^*}{n-1} \\ \implies S &= \frac{VDV^*}{n-1}, \text{ by Theorem 3.} \end{aligned}$$

Result 1 tells us that the eigendecomposition of  $S$  is  $S = \frac{VDV^*}{n-1}$ , because the right singular vectors of  $X$  are the eigenvectors of  $S$ , and the eigenvalues of  $S$  are the squared singular values of  $X$ [3]. Thus, we have a way to compute the eigendecomposition of  $S$  without actually calculating  $S$ , which could be, depending on the dimensions of  $X$ , and extremely intensive computation.

**Algorithms.** Now we want to actually perform the principle component analysis of some data set using Result 1. We do this by performing the steps of Algorithm 1:

*Algorithm 1.* PCA through SVD:

1.) Standardize and mean center the data matrix:

To do this we take each column of our data matrix  $X$  as a vector  $\vec{x}_n$ . Compute the mean,  $\bar{x}$ , and standard deviation,  $s$ , of the entries in  $\vec{x}_n$  and then perform the following computation on each entry of the column,  $x_j$ :

$$z_j = \frac{x_j - \bar{x}}{s},$$

where  $z_j$  is the new entry.

2.) Compute the SVD of  $X$ :

1.) Find the eigenvalues of  $X^*X$ , and find the corresponding eigenvectors. Take the first  $m$  or  $n$  (whichever value is smaller,  $m$  and  $n$  are the dimensions of  $X$ ). Order both the eigenvalues and corresponding eigenvectors in order of descending eigenvalues. The square roots of these eigenvalues you have kept are the singular values that comprise the diagonal of  $\Sigma$ , and the eigenvectors are the columns of  $V$ .

2.) Let the first  $r$  columns of  $U$  be  $\vec{u}_j = \sigma_j^{-1} X \vec{v}_j$ , where  $\vec{v}_j$  is the  $j$ th column of  $V$  and  $r = \text{Rank}(X)$ . Use the Gram-Schmidt process to extend this list of vectors to an orthonormal basis for  $\mathbb{F}^m$ . These are the columns of  $U$ .

3.) Rearrange for eigendecomposition of  $S$ :

Simply compute  $\Sigma^* \Sigma = D$ , and write,

$$S = \frac{VDV^*}{n-1}.$$

4.) Find the feature vector based on chosen tolerance:

Choose a tolerance level with which to judge your eigenvalues. To compute the variance associated with each eigenvalue in the matrix  $D$ , we use:

$$v = \frac{\lambda_j}{\sum_{k=1}^n \lambda_k}$$

If  $v$  is less than our tolerance, then we replace the corresponding eigenvector to  $\lambda_j$  with the zero vector. After doing this for all of the eigenvalues, we call this new matrix  $W$ , which is our feature vector.

5.) Project data onto principle components:

This is done by multiplying  $Z = XW$ . This is the projection of our data  $X$  onto our principle components  $W$ . We can also call  $Z$  the latent representation of the data.

6.) Recover the lower dimension approximation of our data matrix:

This is an optional step depending on the goal of the PCA. To recover the approximation of our data matrix based off the latent representation compute  $X' = ZV^*$ .

Algorithm 1 arrives at the same eigendecomposition of the covariance matrix as many other methods, such as simply computing the eigendecomposition of the covariance matrix straightforwardly. However, it saves the time, processing power, and memory allocation required for computing the covariance matrix of massive data sets. As such, Algorithm 1 is a way to optimize PCA.

## APPLICATION

The applications of PCA are various and sundry, and we could not possibly cover them all in this paper. Rather, we will give a brief overview of one field of applications.

The most relevant application of PCA right now is in the field of machine learning(ML). In this field it is often, almost exclusively, necessary to utilize extremely

large, high-dimension data-sets. PCA serves as a way to one, reduce the dimension of these data sets, and secondly to find trends in this high-dimensional data that would otherwise be impossible to isolate. Thus, it is often useful for clustering models, such as the k-nearest neighbors algorithm.

PCA is also often used in the encoder/decoder structures of Generative Adversarial Networks (GANs) and Recurrent Neural Networks (RNNs). This is done by taking an input tensor, performing PCA on it, generating the latent representation of the matrix, and then reconstructing the matrix from the dimensionally reduced latent representation. The reconstruction error of this matrix is then calculated to score the performance of the model. This reconstruction error value can also be used as a decision threshold for models performing intrusion detection or anomaly detection in cybersecurity.

The PCA through SVD method presented in this paper allows for the optimization of these models. This is extremely valuable for performing analysis on real time data, which is the goal of many ML applications.

## BIBLIOGRAPHY

- [1] Meckes, Elizabeth S., and Mark W. Meckes. *Linear Algebra*. Cambridge University Press, 2018.
- [2] “Eigen Decomposition Theorem.” From Wolfram MathWorld, [mathworld.wolfram.com/EigenDecompositionTheorem.html](https://mathworld.wolfram.com/EigenDecompositionTheorem.html). Accessed 15 May 2024.
- [3] Wilkinson, Prof. Richard. “Multivariate Statistics.” Richard Wilkinson, [richd-wilkinson.github.io/MATH3030/](https://richd-wilkinson.github.io/MATH3030/). Accessed 15 May 2024.